Interview Prep Guide

Evidence-Gated RAG Chatbot with LLM-as-a-Judge Evaluation

Beginner → deep technical explanation, built strictly from your actual code and your actual eval/results.json. Nothing here is invented.

Table of Contents

1. What This Project Is (Beginner Introduction)

In plain terms: this is a chatbot that answers questions about a PDF document you upload — and only about that document. If you ask it something the document doesn't cover, it says so honestly instead of making something up.

This kind of system is called RAG — Retrieval-Augmented Generation. In simple words:

- **Retrieval:** first, search the document for the pieces of text most relevant to the question.
- **Augmented:** give those pieces of text to an AI language model as extra information it wouldn't otherwise have.
- **Generation:** the AI model writes a natural-language answer, using only that provided information.

Why RAG instead of just asking an LLM directly? Because a general-purpose LLM only knows what it was trained on — it has never seen your specific PDF, and if asked about it directly, it would either say it doesn't know or, worse, confidently guess. RAG gives the model the actual document content to work from, and this project goes one step further: it refuses to answer at all if the retrieved content isn't actually relevant, rather than letting the model fall back on guessing.

2. Resume Accuracy Check — Read This First

> **⚠ Important: verify these two details before your interview**
>
> 1. Your resume says "Llama 3." Your CURRENT code uses Groq's openai/gpt-oss-20b, not Llama 3. It genuinely WAS Llama 3 (llama-3.1-8b-instant) for most of the project, until Groq deprecated that model mid-project with a shutdown date only days away, forcing a migration. This is a good story, not a bad one — but say it accurately if asked, and consider updating the resume wording to "Groq-hosted LLMs" so it doesn't read as inaccurate before you get to explain.
>
> 2. Your faithfulness/answer-relevance scores (0.93 / 0.93) are computed over 15 of the 18 golden-set questions, not all 18 — 3 questions were hard-refused, meaning there was no generated answer to judge faithfulness on. The resume phrasing "on an 18-question golden set" is still fair (that IS the size of the golden set), but know the n=15 detail if asked directly.

Full defenses for every resume bullet, with exact numbers, are in Section 12.

3. Technology Stack

Technology	Role in the project
Streamlit	The web UI — file upload, chat interface, displays answers/sources/latency.
LangChain (0.2.x)	Orchestration — prompt templates, the document-stuffing generation chain, message types.
FAISS	Vector similarity search over chunk embeddings.
BM25 (rank_bm25 / langchain-community)	Keyword/lexical search over chunks.
sentence-transformers/all-MiniLM-L6-v2	The embedding model — turns text into vectors for FAISS.
cross-encoder/ms-marco-MiniLM-L-6-v2	The reranking model — scores (question, chunk) pairs jointly.

Groq (langchain-groq)	The LLM provider — fast inference API. Current model: openai/gpt-oss-20b (generation), openai/gpt-oss-120b (evaluation judge only).
pypdf / PyPDFLoader	PDF text extraction, page by page.

4. Complete Query Flow — Step by Step

Ingestion (happens once per uploaded file)

1. File is hashed and saved to disk under a content-hash filename, so re-uploading the same file is instant and different files never collide.
2. PyPDFLoader reads the PDF page by page.
3. Each page's text is split into ~800-character chunks with 120-character overlap.
4. Each chunk gets a unique chunk_id and is embedded into a vector.
5. A FAISS index and a BM25 index are both built over the same chunks.

Per-question flow (every time you ask something)

1. Condense — rewrite the question into a standalone one, but only if it has a referential word AND there's chat history; otherwise skip.
2. Hybrid retrieve — FAISS top-15 + BM25 top-15, merged and de-duplicated.
3. Rerank — the cross-encoder scores every candidate against the actual question.
4. Gate — keep chunks above the primary threshold (up to 4); if none qualify, keep just the single best chunk if it's above a much looser floor; if not even that, refuse immediately with no LLM call.
5. Generate — the LLM writes an answer using only the chunks that survived the gate, at temperature=0, with a prompt that forbids outside knowledge.
6. Leak check — scan the answer for phrases suggesting outside knowledge leaked in; if found, regenerate once with a stronger reminder.
7. Display — show the answer, the exact source chunks used (with their rerank scores), and the time each stage took.

5. Deep Dive: Every Component Explained

PDF Loading & Chunking

What it is: The process of taking a PDF file and breaking it into small, searchable pieces of text called “chunks.”

Why it's used: An LLM can't read an entire PDF at once (context limits, cost, and precision all suffer). Breaking the document into small pieces lets the system search for and retrieve only the specific pieces relevant to a question.

How it works: PyPDFLoader reads the PDF one page at a time (each page becomes a separate object with its own text and page number). Then RecursiveCharacterTextSplitter cuts each page's text into pieces of about 800 characters, with 120 characters of overlap between consecutive pieces so a sentence split across a chunk boundary isn't completely lost. Chunks are never allowed to cross a page boundary — splitting happens independently within each page.

My implementation: In src/document_loader.py, load_and_split_pdfs() does this exact sequence: load pages → clean text (collapse whitespace, remove null bytes) → split with RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=120) → discard any chunk under 30 characters (MIN_CHUNK_LENGTH, filters out stray headers/page numbers) → assign each surviving chunk a chunk_id like 'JD-Analyst.pdf::p1::0' (filename :: page :: index).

Simple example: *A 3-page job description PDF in my project produced exactly 5 chunks: 2 chunks from page 1, 2 from page 2, 1 from page 3 — verified by literally running the chunking code against the real file and printing the output.*

Embeddings

What it is: A way of converting a piece of text into a list of numbers (a "vector") that captures its meaning, so that pieces of text with similar meaning end up with similar numbers.

Why it's used: Computers can't compare the "meaning" of two sentences directly — but they can compare two lists of numbers very fast. Embeddings let the system find chunks that mean something similar to the question, even if they don't use the exact same words.

How it works: A pretrained neural network (the embedding model) reads a piece of text and outputs a fixed-length vector (for this model, 384 numbers). Texts about similar topics end up with vectors that point in similar directions in that 384-dimensional space.

My implementation: sentence-transformers/all-MiniLM-L6-v2 (via HuggingFaceEmbeddings in LangChain) is used to embed every chunk once at indexing time, and to embed each user question at query time. This is set in src/config.py as EMBEDDING_MODEL and loaded (and cached, so it's not reloaded every request) in get_embeddings() in src/rag_engine.py.

Simple example: *The chunk containing "Curious and a passion for data-driven problem solving" and the question "What soft skills are mentioned?" end up as vectors that are close together, even though they don't share many exact words.*

FAISS (Vector Search)

What it is: A library (from Meta) for storing a large collection of vectors and quickly finding the ones most similar to a given query vector.

Why it's used: Once every chunk is embedded, you need a fast way to find the chunks whose vectors are closest to the question's vector — comparing against every chunk one by one would be slow at scale, and FAISS is built specifically for this.

How it works: FAISS builds an index over all the chunk vectors. Given a query vector, it computes similarity (distance) to every stored vector and returns the top-k closest ones — this is "semantic" search, based on meaning, not exact word matches.

My implementation: In src/rag_engine.py, build_indexes() creates FAISS.from_documents(chunks, embeddings) once per uploaded file (cached by content hash, so re-uploading the same file doesn't rebuild it, but a genuinely different file always does). At query time, vectorstore.similarity_search(query, k=15) returns the top 15 semantically closest chunks.

Simple example: *Asking "What will I gain from this job?" retrieves the chunk under the heading "What You'll Gain" even though the question doesn't use that exact phrase — because their meanings are close in vector space.*

BM25 (Lexical/Keyword Search)

What it is: A classic, decades-old information-retrieval algorithm that scores documents by exact keyword overlap with the query — no neural network involved.

Why it's used: Embeddings are great at meaning but can miss exact terms: specific names, IDs, acronyms, or technical terms that a keyword search catches instantly and for free. Combining both is how the project avoids each method's individual blind spot.

How it works: BM25 counts how often query words appear in each chunk, weighted by how rare/important those words are across the whole document, and ranks chunks by that score.

My implementation: BM25Retriever.from_documents(chunks) is built alongside the FAISS index in build_indexes() (src/rag_engine.py), with its k set to TOP_K_RETRIEVE (15) via bm25_retriever.k = config.TOP_K_RETRIEVE.

Simple example: *Asking "Are AWS and GCP mentioned?" — BM25 finds the exact chunk containing the literal strings "AWS" and "GCP" directly, which is a very reliable, fast win for exact-term questions.*

Hybrid Retrieval (merging FAISS + BM25)

What it is: Running both the vector search (FAISS) and the keyword search (BM25) for every question, then combining their results into one candidate list.

Why it's used: Neither method alone is complete — vector search can miss exact terms, keyword search can miss paraphrased questions. Combining them and letting a later, more precise step (reranking) sort out the winner gives the best of both.

How it works: Each method independently returns its own top-15 candidate chunks. The two lists are merged and any chunk appearing in both is only kept once (deduplicated by chunk_id).

My implementation: hybrid_retrieve() in src/rag_engine.py: calls vectorstore.similarity_search() and bm25_retriever.invoke(), combines both lists into a dictionary keyed by chunk_id (which automatically removes duplicates), and returns the combined list — typically fewer than 30 chunks since there's real overlap between the two methods.

Simple example: *For "Which cloud platforms are mentioned?", FAISS might find the chunk via general topic similarity while BM25 finds it via the literal words "cloud platforms" — same chunk from both, deduplicated into one entry, still ranked highly.*

Cross-Encoder Reranking

What it is: A second-stage, more expensive but far more precise model that re-scores each candidate chunk against the actual question, one pair at a time.

Why it's used: FAISS and BM25 both score a query and a chunk independently and then compare the scores — that's fast but has a real precision ceiling. A cross-encoder reads the query and the chunk together in one pass, which lets it judge relevance far more accurately — too slow to run over an entire document, but fast enough to run over the ~30 candidates the hybrid step already narrowed things down to.

How it works: The cross-encoder cross-encoder/ms-marco-MiniLM-L-6-v2 takes (question, chunk) pairs and outputs a relevance score for each — an unbounded number (not a 0-1 probability), where higher means more relevant.

My implementation: rerank() in src/rag_engine.py builds a (query, chunk_text) pair for every hybrid-retrieved candidate, calls reranker.predict(pairs), and sorts the results best-score-first. The model is loaded once and cached via get_reranker().

Simple example: *For "What are Key Responsibilities?", the chunk literally headed "Key Responsibilities" might score 5.5+, while a chunk that's only vaguely related scores around 0 or negative — the reranker is what actually separates "relevant" from "vaguely similar."*

Score Gating — the "Grounded-or-Refused" Policy

What it is: A hard rule, enforced in code before the LLM is ever called for generation, that decides whether there's actually enough relevant information to answer the question at all.

Why it's used: Without this, a chatbot will happily answer any question using its own general training knowledge even when the uploaded document says nothing about it — which defeats the entire purpose of a document-specific assistant and risks the user trusting an answer that isn't actually backed by their document.

How it works: Two tiers: (1) Primary threshold — keep any reranked chunk scoring ≥ RERANK_SCORE_THRESHOLD (−2.0), up to 4 chunks (TOP_K_RERANK). (2) Floor fallback — if nothing clears that primary bar, but the single best-scoring chunk is still ≥ RERANK_FLOOR_SCORE (−6.0, a much looser bar), serve just that one chunk instead of refusing outright. If even that floor isn't cleared, the system refuses — and critically, it never even calls the LLM to generate an answer in that case, so there's no chance of it improvising one.

My implementation: This logic lives directly inside answer_question() in src/rag_engine.py. If nothing survives either tier, the function returns config.REFUSAL_MESSAGE immediately — no generation call happens at all, which also saves cost and latency on unanswerable questions.

Simple example: *Asking "What color is the company logo?" against a job-description PDF: no chunk is remotely relevant, nothing clears even the floor score, so the app replies "I couldn't find this in the uploaded document(s)" without ever asking the LLM to guess.*

Conversational Memory

What it is: The ability to understand follow-up questions that depend on earlier turns in the conversation ("What about that?", "And the second one?") by rewriting them into a standalone question before retrieval.

Why it's used: Retrieval search needs a self-contained question to work with — searching for "what about that?" on its own would find nothing useful. Rewriting it using chat history ("What about the salary for the Analyst role?") makes retrieval work correctly across a multi-turn conversation.

How it works: An LLM call rewrites the follow-up into a standalone question, using the prior conversation as context. But this only happens when actually needed: a simple keyword check first looks for referential words (it, that, this, second, previous, etc.) — if the question is already self-contained, the rewrite step is skipped entirely, both to save an LLM call and, importantly, because letting the LLM rewrite an already-clear question was found (during testing) to sometimes drift the question's wording and hurt retrieval accuracy.

My implementation: condense_question() and the _needs_condensing() heuristic in src/rag_engine.py. _needs_condensing() checks the question against a fixed list of referential marker words; condense_question() only calls the LLM if that check is true and there is prior chat history.

Simple example: *"What are the key responsibilities?" followed by "What about under Data Science?" — the second question has no referential word ("Data Science" is a real topic name, not a pronoun), so this actually gets treated as already self-contained and searched as-is, which turned out to work correctly without needing a rewrite.*

Generation (Groq LLM)

What it is: The step where an LLM actually writes the final answer, using only the chunks that survived the score gate as its allowed source of information.

Why it's used: This is where the retrieved information gets turned into a natural-language answer a human can read — but it has to be constrained to only use what was actually retrieved, or the whole grounding guarantee falls apart.

How it works: The served chunks are "stuffed" into a prompt alongside the question, with a system prompt that explicitly forbids using any outside knowledge and tells the model to say plainly when the context only partially covers the question, rather than filling gaps by guessing.

My implementation: get_document_chain() in src/rag_engine.py builds this using LangChain's create_stuff_documents_chain, with the model set via get_llm() in src/llm.py. Currently using Groq's openai/gpt-oss-20b at temperature=0 (deterministic — chosen after observing the same question get answered differently across repeated runs at non-zero temperature). model_kwargs also sets reasoning_format="hidden" so the model's internal step-by-step reasoning never leaks into the visible answer text.

Simple example: *Given the chunk containing "Analyze large volumes of structured and unstructured data to extract meaningful business insights," the model answers exactly that — it does not add unstated details about, say, what tools are typically used for that in general industry practice.*

Leak Detection & Retry

What it is: A safety-net check on the generated answer text itself, looking for phrases that suggest the model slipped outside knowledge in anyway, even with the strict prompt and temperature=0.

Why it's used: No prompt is 100% foolproof against a model occasionally hedging with "based on general knowledge…" — this is a second, independent layer of defense rather than relying on the prompt alone.

How it works: After generation, the answer text is scanned for phrases like "based on general knowledge," "typically," "as far as I know." If found, the same question is sent back to the model exactly once more, with an extra, more forceful reminder appended, asking it to answer using only the given context.

My implementation: config.LEAK_PHRASES holds the phrase list; _contains_knowledge_leak() checks for them; answer_question() in src/rag_engine.py does the one-time regenerate-with-reminder if a leak is detected.

Simple example: *If the model answers "Based on general knowledge, soft skills usually include…" instead of quoting the document's actual Soft Skills section, the leak check catches the phrase, and the system asks again with a stricter reminder before showing anything to the user.*

LLM-as-a-Judge (Evaluation Only)

What it is: Using a second, separate LLM to score the quality of the generated answers — not during normal chatbot use, only when running the evaluation pipeline.

Why it's used: Metrics like "was this answer actually faithful to the source?" or "did it actually address the question?" can't be measured with simple string matching — they require judgment. Rather than trust a general-purpose external library, this project uses a small, purpose-built judge module that only depends on the same Groq client the app already uses.

How it works: Three judge prompts, each asking for a strict JSON response so the score can be parsed reliably: (1) Faithfulness — the judge breaks the generated answer into individual factual claims and checks each one against the retrieved context, returning supported_claims / total_claims. (2) Answer relevance — the judge rates 0.0–1.0 how directly the answer addresses the question asked. (3) Soft-refusal detection — for the specific case where a question was truly unanswerable but the hard refusal gate didn't fire, checks whether the model's own generated answer amounts to an honest "not in the document" admission.

My implementation: eval/llm_judge.py defines judge_faithfulness(), judge_answer_relevance(), and judge_is_soft_refusal(). All three use a separate, stronger model (JUDGE_MODEL = openai/gpt-oss-120b) than the one that generated the answers (GROQ_MODEL = openai/gpt-oss-20b) — this was a deliberate fix after the small model, used as its own judge, gave visibly unreliable scores (clearly correct answers scored 0.0 for relevance). If a judge call fails or the response can't be parsed as valid JSON, the score is recorded as None, never guessed as 0 or 1.

Simple example: *For the answer "AWS and GCP (Amazon Web Services and Google Cloud Platform)" against a source that only says "AWS, GCP," the faithfulness judge correctly flags the acronym expansions as unsupported claims, scoring that answer 0.5 instead of 1.0.*

The Evaluation Pipeline

What it is: A separate, reproducible script (not part of the live chatbot) that runs a fixed, hand-written set of test questions through the real production pipeline and reports measurable numbers.

Why it's used: "It seems to work when I chat with it" isn't evidence — a fixed golden set that's run the same way every time is what lets you compare before/after a change and prove whether something actually got better or worse.

How it works: For every golden-set question: run it through the exact same hybrid_retrieve → rerank → gate → generate pipeline the live app uses (not a separate simplified version), then compare the retrieved chunk_ids against hand-labeled "correct" chunk_ids to compute retrieval metrics, and (optionally) call the LLM judges to score faithfulness/relevance.

My implementation: eval/run_eval.py is the runner. eval/metrics.py computes Recall@K, Precision@K, MRR (Mean Reciprocal Rank), Hit Rate, and refusal_quality using pure set arithmetic — no LLM involved, so these are exact, not estimated. eval/golden_set.json holds the 18 hand-written questions, each with its correct chunk_id(s) verified directly against the extracted PDF text (not guessed from the UI).

Simple example: *Running python eval/run_eval.py --golden-set eval/golden_set.json --pdf-dir eval/sample_pdf --llm-judge produces eval/results.json with every metric below, computed fresh, every time.*

6. Important Files & Functions

File	What to know
src/config.py	All tunable constants: CHUNK_SIZE/CHUNK_OVERLAP, TOP_K_RETRIEVE/TOP_K_RERANK, RERANK_SCORE_THRESHOLD/RERANK_FLOOR_SCORE, LLM_TEMPERATURE, GROQ_MODEL vs JUDGE_MODEL, LEAK_PHRASES.
src/document_loader.py	load_and_split_pdfs() — PDF loading, chunking, chunk_id assignment.
src/llm.py	get_llm() — builds the Groq client, sets temperature=0 and reasoning_format=hidden.
src/rag_engine.py	The core engine: hybrid_retrieve(), rerank(), condense_question(), answer_question() (the full per-query pipeline), leak detection.
app.py	The Streamlit UI — file upload, chat interface, calls answer_question(), displays sources/latency, refuses to run without an uploaded PDF (no general-chat fallback).
eval/run_eval.py	The evaluation harness — runs the golden set through the real pipeline, computes all metrics, calls the LLM judges.
eval/metrics.py	Exact-match Recall@K/Precision@K/MRR/Hit Rate/refusal_quality — no LLM involved.
eval/llm_judge.py	judge_faithfulness(), judge_answer_relevance(), judge_is_soft_refusal().
eval/golden_set.json	The 18 hand-labeled test questions and their correct chunk_ids/answers.

7. Your Actual Results — Explained

Retrieval quality

Metric	Raw (pre-rerank)	Served (post-rerank + gate)
Recall@K	1.00	0.88
Precision@K	0.29	0.87
MRR	0.81	0.93
Hit Rate	1.00	1.00

Reranking took precision from 29% to 87% — nearly a 3x improvement — while recall only dropped slightly and hit rate stayed perfect. This is the clearest, most quotable proof that reranking is doing real work.

Refusal quality

Metric	Hard-gate only	Effective (incl. honest declines)
Refusal recall	0.75	1.00
False refusal rate	0.00	0.00

Zero false refusals in both variants — the system never wrongly declined a question it could actually answer.

Generation quality (LLM-as-judge)

Metric	Mean	Median	n
Faithfulness	0.93	1.00	15
Answer relevance	0.93	1.00	15

Latency

Stage	Median	p95
Retrieval	30ms	52ms
Rerank	217ms	292ms
Generation	560ms	736ms
Total	788ms	1038ms

8. Real Challenges Faced (and How They Were Fixed)

1. A refusal threshold that was too strict

The very first version used a single hard cutoff (0.0) for the rerank score gate. Testing found it correctly blocked unrelated questions but also wrongly refused real, present content whenever a question was phrased differently from the document's exact wording. Fixed with the two-tier gate (primary threshold + a much looser floor fallback).

2. A grounding leak despite a strict prompt

Even with instructions not to use outside knowledge, the model occasionally added phrases like “based on general knowledge…” This was fixed with three combined changes: temperature=0 for determinism, a stricter prompt explicitly forbidding this, and a post-generation phrase check that triggers one automatic retry with a stronger reminder.

3. Query drift from over-aggressive follow-up rewriting

Originally, every follow-up question was rewritten by an LLM using chat history, even when it didn't need it — and this was found to sometimes drift the question's meaning and break retrieval for what was really a simple, already-clear question. Fixed by only rewriting when the question actually contains a referential word (it, that, second, etc.).

4. RAGAS dependency conflicts (twice)

First attempt: RAGAS needed a modern langchain-core incompatible with the pinned stack. Second attempt, in a fully separate virtual environment: RAGAS itself had an unrelated, upstream bug (an import to a class relocated in newer langchain-community). Resolved by building a small, dependency-free LLM-judge module instead of depending on RAGAS at all.

5. An unreliable self-judging model

Using the small production model to judge its own answers produced clearly wrong scores (correct answers scored 0.0 relevance). Fixed by using a separate, stronger model purely for judging.

6. A mid-project model deprecation

The original production model (llama-3.1-8b-instant) was deprecated by Groq with a shutdown date only days away. Migrated to openai/gpt-oss-20b, and specifically had to add reasoning_format="hidden" since the new model is a reasoning model that could otherwise leak its internal step-by-step thinking into the visible answer.

7. Golden-set labeling errors, caught by verification

Early golden-set entries assumed which chunk contained the answer to a question based on what the app happened to retrieve — which was circular reasoning (the app's own behavior isn't ground truth). Fixed by running the actual chunking code directly against the real PDF and verifying every label against the real extracted text.

8. A refusal-quality metric blind spot

The hard refused flag only counted the canned refusal message, so a case where the model honestly said "the document doesn't specify this" (without the hard gate firing) wasn't being counted as safe behavior even though it was. Added a second, LLM-judged "effective refusal" metric to capture this.

9. "Why X Instead of Y?" — Design Decisions

Q: Why hybrid retrieval instead of vector search alone?

A: Because embeddings can miss exact terms — IDs, names, acronyms — that a plain keyword search catches for free. BM25 costs almost nothing to add and covers that specific blind spot.

Q: Why add reranking if you already retrieve top-k?

A: Because a bi-encoder (what both FAISS and BM25 effectively are) scores query and document independently, which has a real precision ceiling. A cross-encoder scores them jointly — more accurate, but too slow to run over a whole corpus, so it's used only on the smaller candidate pool retrieval already narrowed down.

Q: Why refuse instead of letting the LLM answer from general knowledge?

A: Because a document-specific assistant that quietly answers from outside knowledge isn't trustworthy — the user can't tell which answers actually came from their document. This was made a hard, code-enforced rule (a score gate before generation), not just a prompt instruction.

Q: Why a two-tier gate instead of one single threshold?

A: A single hard threshold was tried first and found (via the eval harness) to correctly block unrelated questions but also wrongly refuse real content that a paraphrased question scored just under the bar. The floor fallback recovers those cases without opening the door to genuinely unrelated content.

Q: Why temperature=0 instead of the default?

A: For determinism. The same question was observed to get answered correctly on one run and either refused or padded with outside knowledge on the next, purely from random sampling — several real bugs traced back partly to this.

Q: Why build a custom LLM-judge instead of using RAGAS?

A: RAGAS's dependencies required a much newer langchain-core than the pinned version langchain-groq needed — confirmed incompatible by testing, not assumed. A second attempt, in a fully separate environment, still hit an unrelated bug inside RAGAS itself (a hardcoded import to a class that had been relocated in a newer langchain-

community). Rather than keep fighting someone else's packaging issues, a small, dependency-free judge module was built instead.

Q: Why use a different (bigger) model as the judge than the one generating answers?

A: The production model, used to judge its own answers, gave visibly unreliable scores — clearly correct, on-topic answers sometimes scored 0.0 for relevance. Small models aren't reliable at self-rating a continuous 0–1 scale. A separate, stronger model fixed this immediately.

Q: Why chunk_size=800 with 120 overlap, not smaller or bigger chunks?

A: A smaller size (originally 400 in an earlier version of this project) tends to fragment sentences and hurt retrieval quality with a small embedding model. 800 with overlap keeps a chunk coherent while still comfortably fitting in the LLM's context window.

Q: Why not use LangChain's built-in create_retrieval_chain / history_aware_retriever?

A: Those higher-level chains hide the retrieval results and scores inside the chain — there's no way to inspect a chunk's rerank score and decide whether to refuse before generation happens. The pipeline was written explicitly (condense → retrieve → rerank → gate → generate) specifically so the refusal decision could be made with full visibility into the scores.

10. Implemented Features vs. Future Improvements

Implemented (verified in the actual code and eval results)

- Hybrid retrieval (FAISS + BM25), merged and deduplicated
- Cross-encoder reranking
- Two-tier score gate enforcing grounded-or-refused
- Conversational memory with a referential-word bypass to avoid unnecessary query rewriting
- Post-generation leak detection with one automatic retry
- Custom, dependency-free LLM-as-judge module (faithfulness, answer relevance, soft-refusal detection)
- Reproducible evaluation harness (Recall@K, Precision@K, MRR, Hit Rate, refusal quality, latency)
- Multi-PDF upload support in the app (though the golden set has only been evaluated against one document so far)
- Successful mid-project migration off a deprecated LLM

Future improvements (NOT implemented — do not claim these)

- Per-claim or per-sentence citations (currently citations are at the chunk level only)
- A persisted vector store across sessions (currently rebuilt per session, cached only within that session)
- GPU-accelerated reranking (currently CPU-only, and reranking is the largest retrieval-side latency cost)
- A larger, multi-document golden set (currently 18 questions against a single PDF)
- Formal calibration of the gate thresholds (currently tuned from observed failure cases, not a grid search or statistical study)

11. Interview Questions, Answers, and Follow-Ups

Interviewer: Walk me through what happens when a user asks a question.

You: The question first goes through a quick check: if it has referential words and there's chat history, an LLM rewrites it into a standalone question, otherwise it's used as-is. Then it's searched two ways at once — FAISS for semantic similarity and BM25 for exact keyword matches — and the results are merged into one candidate list. A cross-encoder then re-scores every candidate against the actual question. Chunks that clear a relevance threshold (or,

as a fallback, just the single best chunk if it's at least weakly relevant) get passed to the LLM, which generates an answer using only those chunks. If nothing is relevant enough, the system refuses before ever calling the LLM to generate anything.

Likely follow-up: What happens if two different chunks say slightly different things?

Interviewer: What's the difference between recall_raw and recall_served in your evaluation?

You: recall_raw measures the hybrid retrieval candidate pool before reranking — basically, did the correct chunk exist anywhere in what was found. recall_served measures what's left after reranking and the score gate — what actually got shown to the LLM. Comparing the two isolates exactly what the reranker contributed.

Likely follow-up: In your results, recall dropped slightly from raw to served — doesn't that mean the reranker made things worse?

Interviewer: How do you know your evaluation numbers aren't just made up?

You: Every metric is computed by code, not asserted. Recall@K/Precision@K/MRR/Hit Rate come from exact set comparisons against a hand-labeled golden set — no LLM involved, so they're exact. Faithfulness and relevance come from an actual LLM call at evaluation time; if that call fails or returns something unparseable, the result is recorded as None, never defaulted to a number.

Likely follow-up: How big is your golden set, and is that enough to trust these numbers?

Interviewer: Why does your project refuse some questions instead of always answering?

You: Because the core requirement was that the chatbot only ever answers from the actual uploaded document — never its own general knowledge. If nothing relevant enough was retrieved, refusing is the honest answer; making something up would be worse than saying 'I don't know.'

Likely follow-up: How did you verify the refusal behavior is actually correct, not just present?

Interviewer: What would break this system, or where are its current weaknesses?

You: When a document's section spans two chunks (because of the fixed 800-character chunk size), the reranker sometimes only serves one of them, giving a technically correct but incomplete answer — never a wrong or hallucinated one, just partial. The golden set is also currently small (18 questions, one document), which is enough to have already caught three real bugs but not enough to claim statistically tight numbers.

Likely follow-up: How would you fix the chunk-splitting issue?

12. Resume Bullet-by-Bullet Defense

Resume bullet: *"Developed a document-grounded RAG chatbot using LangChain, FAISS, BM25, HuggingFace, Llama 3, and Groq..."*

How to defend it: All accurate except one word: the model is currently Groq's openai/gpt-oss-20b, not Llama 3. It WAS Llama 3 (llama-3.1-8b-instant) for most of development, until Groq deprecated it mid-project. If asked directly: "It was originally Llama 3 via Groq; the model was deprecated by the provider partway through, so I migrated to Groq's gpt-oss-20b and verified quality held afterward using my own evaluation pipeline." This is a strong story to tell, not something to hide — but the resume wording should ideally be updated to say "Groq-hosted LLMs" or name the current model, so it doesn't read as inaccurate before you get the chance to explain.

Resume bullet: *"...improving retrieval precision from 29% to 87% while maintaining 100% hit rate."*

How to defend it: Directly from eval/results.json: retrieval_raw precision@K = 0.2857 (~29%), retrieval_served precision@K = 0.8690 (~87%), hit_rate = 1.00 in both cases. Be ready to explain raw vs served (see the interview Q&A above) — that's the first thing an interviewer will probe on this bullet.

Resume bullet: *"Built an LLM-as-a-Judge evaluation module, achieving 0.93 faithfulness and 0.93 answer relevance on an 18-question golden set."*

How to defend it: Directly from eval/results.json: generation_quality.faithfulness.mean = 0.9333, answer_relevance.mean = 0.93. One precise detail to know: these are computed over n=15, not all 18 — 3 questions

were hard-refused (nothing to judge faithfulness on) and 1 more, though unanswerable, produced a generated answer that was still judged. If asked “why 15 not 18,” that's the exact reason.

Resume bullet: *“...measuring Recall@K, Precision@K, MRR, Hit Rate, refusal quality, and latency, achieving 788 ms median response latency.”*

How to defend it: Directly from eval/results.json: latency_ms.total.median = 787.79ms (≈788ms), p95 = 1038ms. All five named metrics (Recall@K, Precision@K, MRR, Hit Rate, refusal quality) are genuinely computed by eval/metrics.py — this bullet is fully accurate as written.